Delivery Area & Spatial Search System

A step-by-step guide to defining serviceable delivery areas and searching them efficiently, from polygon boundaries to spatial indexing.

The system stores the polygon in PostgreSQL/PostGIS

1. Core Concepts
1.1. Latitude / Longitude [01]
1.2. GeoJSON [03]
1.3. WKT [04]
1.4. PostGIS [04]
1.5. JTS [05]

2. Interactive Maps with Plain HTML & JS
2.1. Add Leaflet: [07]
2.2. Add OpenStreetMap tiles: [08]
2.3. Enable Polygon [08]
2.4. Capture Drawn Polygon [08]
2.5. Convert Polygon to GeoJSON [08]
2.6. Send Polygon to Backend [08]

3. Interactive Maps with React
3.1. Understand Tile URL [09]
3.2. Custom Marker Icon [10]
3.3. Current Location [10]
3.4. Drawing a Polyline(Line / Road) & Polygon [10]
3.5. Drawing Tools [11]

4. GeoJSON(Leaflet → API Format)
4.1. Geometry vs Geography [09]
4.2. Validate Polygon [10]

5. Store Polygon using JTS [14]
6. Store Polygon using WKT [14]
7. PostGIS → GeoJSON [15]
- ST_Contains / ST_Within [15]
- ST_Covers / ST_Area [15]
- ST_DWithin / ST_Distance [15]

8. GoogleMap Equivalent [17]
9. Create Spatial Index [18]
10. Store GeoHash Reference [19]
- GeoHash Prefilter [19]
- KD Tree [19]

11. Dynamic Delivery Zone [21]
- Traffic-Aware Zones
- Driver Availability
- Weather-Aware Zones
- Demand-Aware Zones

1. Core Concepts

1.1. Latitude

Latitude → North / South (**90° N - 90° S**)
Longitude → East / West (**180° E - 180° W**)

Imagine Earth has a line around its middle: the **Equator**.

- Equator = **0° latitude**
- North Pole = **90° N**
- South Pole = **90° S**

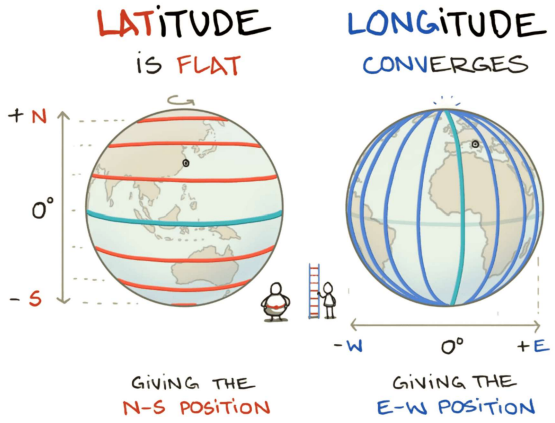
So latitude tells you **north ↕ south**.

For example:

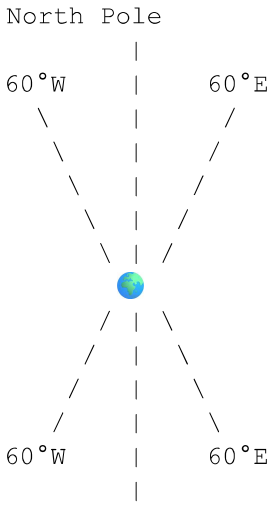
- Hyderabad ≈ **17.4° N**
- New York ≈ **40.7° N**
- Sydney ≈ **33.9° S**

The higher the latitude, the farther you are from the Equator.

```
            North Pole
               90° N
                |
               60° N
                |
               30° N
                |
Equator ------ 0° ------
                |
               30° S
                |
               60° S
                |
               90° S
            South Pole
```



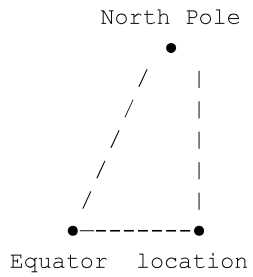
Longitude



Hyderabad: 17.4° N, 78.5° E
1. Go **17.4° north** of the Equator.
2. Go **78.5° east** of the Prime Meridian.
3. Where those two lines intersect → **Hyderabad**.

But what does "17.4 degrees" actually mean?

This is where it gets interesting.Earth is approximately a sphere, so we measure angles from the center of Earth.



If you draw a line from the **center of Earth** to a point on the surface, the angle relative to the Equator determines the latitude.

So **latitude is fundamentally an angle**.

Longitude is also an angle, but measured **around Earth's rotation axis**.

Why GPS uses them

Your phone doesn't literally "see" latitude and longitude.

GPS satellites send extremely accurate **time signals**. Your phone compares when those signals arrived from several satellites and calculates its distance from each one.

With enough satellites, it can determine a point in 3D:

latitude + longitude + altitude

Then your mapping app takes those coordinates and says:

One subtle thing: **latitude lines are parallel**, but longitude lines converge at the poles. That's why a degree of longitude represents different physical distances depending on where you are on Earth. A degree of latitude is roughly **111 km**, while a degree of longitude is about **111 km × cos(latitude)**.

1.2. GeoJSON

GeoJSON is a standard format for representing geographic data as JSON.
Supports Multiple Shapes: It can represent basic geometric shapes and combinations:

- **Point:** Single locations like stores or addresses.
- **LineString:** Connected paths like roads or rivers.(a path/road/route)
- **Polygon:** Enclosed areas like parks or country borders.
- **Multi-part collections:** Groupings like MultiPoint or FeatureCollection.

GeoJSON normally uses: [longitude, latitude]

Not: [latitude, longitude]

Example1:

```
{
   "type": "Point",
   "coordinates": [78.4867, 17.3850]
}
```

Example2:

```
{
  "type": "Feature",
  "geometry": {
    "type": "Polygon",
    "coordinates": [[
      [78.48, 17.38],
      [78.50, 17.38],
      [78.50, 17.40],
      [78.48, 17.40],
      [78.48, 17.38]
    ]]
  }}
```

Why does GeoJSON use longitude, latitude?

Because GeoJSON follows the **coordinate-axis convention**:
In normal geometry, we write: [X, Y] -> For example [10, 20]
where:
  X = horizontal position →
  Y = vertical position ↑
Geographic coordinates can be mapped onto that idea:
  X = longitude
  Y = latitude
Therefore GeoJSON uses: [longitude, latitude]

1.3. WKT

WKT stands for **Well-Known Text**. It's another standard way of representing geographic data, similar in purpose to GeoJSON.

> **GeoJSON = geographic data represented using JSON**
> **WKT = geographic data represented using text**

POLYGON((78.48 17.38, 78.50 17.38, 78.50 17.40, 78.48 17.40, 78.48 17.38))	POINT (78.4867 17.3850)	LINESTRING (78.4800 17.3800, 78.4900 17.3850, 78.5000 17.3900)

1.4. PostGIS

PostGIS is a PostgreSQL extension for geographical data.

PostGIS is an open-source extension for the PostgreSQL relational database that adds support for storing, indexing, and querying geographic and spatial data

Key Features

- **Spatial Data Types:** Adds support for geometric and geographic shapes like points, lines, polygons, and multi-part collections.
- **Spatial Indexing:** Uses R-tree spatial indexes so the database can filter locations quickly without scanning every single row.
- **Spatial Functions:** Provides hundreds of SQL functions to calculate distances, measure areas, perform buffer zones, do spatial joins, and transform coordinates.
- **Raster Support:** Allows storage and analysis of grid-based data like satellite imagery, elevation models, and weather data. [1, 2, 3, 4, 5]

1.5. JTS (Java Topology Suite)

JTS stands for **Java Topology Suite**. It's a Java library for working with **geometric and spatial data**. For example, JTS can understand:

JTS provides algorithms for reasoning about these geometric relationships.

> **POINT (78.4867 17.3850)**

and perform operations such as:

- Is this point inside this polygon?
- Do these two shapes intersect?
- What's the distance between these geometries?
- What's the boundary of this shape?
- What's the intersection between two areas?
- Create a buffer around a geometry.
- Combine or subtract geometries.

You can use JTS's `WKTReader` to turn that text into an actual Java `Geometry` object.

Dependency:

```
<dependency>
    <groupId>org.locationtech.jts</groupId>
    <artifactId>jts-core</artifactId>
</dependency>
```

```
WKTReader reader = new WKTReader();
Geometry geometry = reader.read("POINT
(78.4867 17.3850)");
```

Why is it called "Topology" Suite?

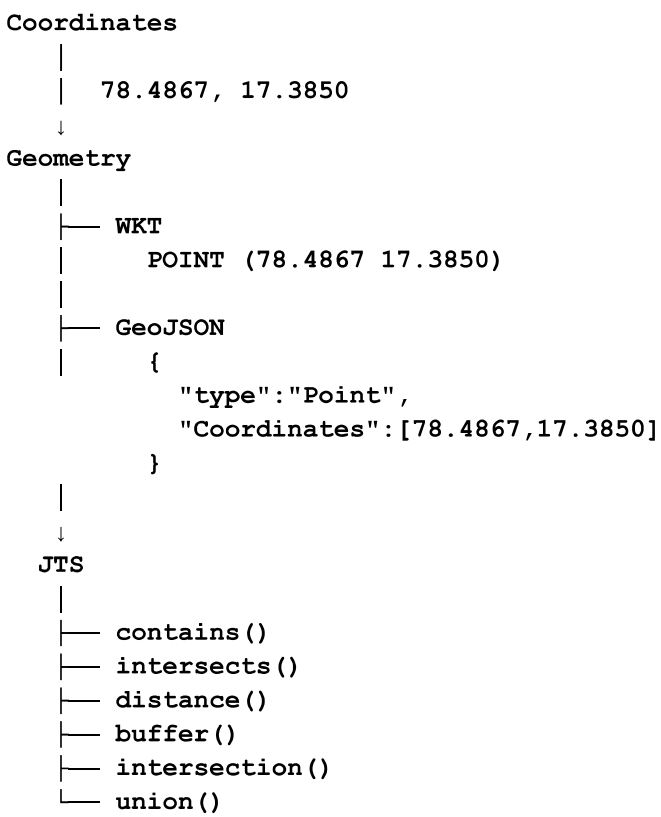
Topology is the branch of mathematics concerned with properties of shapes that remain meaningful when shapes are stretched or transformed.

In GIS, topology is important for relationships such as:

> **Does A touch B?**
> **Does A overlap B?**
> **Does A contain B?**
> **Do A and B intersect?**

The big picture

JTS is **not a mapping application**, Instead, it's a **geometry engine**.



In one sentence: JTS is a Java geometry engine that lets your Java application parse, create, manipulate, and analyze spatial geometries.

2. Interactive Maps with Plain HTML & JavaScript

OpenStreetMap	**Google Maps**
A common setup is: OpenStreetMap ↓ Map tiles ↓ Leaflet ↓ React application **Leaflet = JavaScript map UI library**	Google provides both the map platform and JavaScript APIs. Typical architecture: React ↓ Google Maps JavaScript API ↓ Google Maps platform

STEP 1 — Create the Map:

```
<div id="map"></div>
```

```
const map = L.map('map')
    .setView([17.385, 78.486], 13);
```

STEP 2 — Add Leaflet:

```
<link rel="stylesheet" href="https://unpkg.com/leaflet/dist/leaflet.css" />
<script src="https://unpkg.com/leaflet/dist/leaflet.js"/></script>
```

Add OpenStreetMap tiles:

```
L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png',{
        maxZoom: 19,
        center=[17.3850, 78.4867],
        style={{ height: "100vh", width: "100%" }}
}).addTo(map);
```

STEP 3 — Enable Polygon Drawing:

3.1. Use Leaflet Draw:

```
<link rel="stylesheet"
href="https://cdnjs.cloudflare.com/ajax/libs/leaflet.draw/1.0.4/leaflet.draw.css"
/>

<script
src="https://cdnjs.cloudflare.com/ajax/libs/leaflet.draw/1.0.4/leaflet.draw.js">
</script>
```

3.2. Create a feature group:

```
const drawnItems = new L.FeatureGroup();
map.addLayer(drawnItems);
```

3.3. Add controls:

```
const drawControl = new L.Control.Draw({
    edit: {
        featureGroup: drawnItems
    },

    draw: {
        polygon: true,
        rectangle: false,
        circle: false,
        marker: false,
        polyline: false
    }
});
map.addControl(drawControl);
```

STEP 4 — Capture Drawn Polygon:

```
map.on(
    L.Draw.Event.CREATED,
    function(event) {
        const layer = event.layer;
        drawnItems.addLayer(layer);

        const coordinates = layer.getLatLngs()[0];
        console.log(coordinates);
    }
);
```

Example:

```
[
    {lat: 17.385, lng: 78.486},
    {lat: 17.390, lng: 78.492},
    {lat: 17.380, lng: 78.500}
]
```

STEP 5 — Convert Polygon to GeoJSON:

Leaflet can directly convert the polygon:

```
const geojson = layer.toGeoJSON();
console.log(geojson);
```

Result:

```
{
  "type": "Feature",
  "geometry": {
    "type": "Polygon",
    "coordinates": [...]
  }
}
```

STEP 6 — Send Polygon to Backend:

```
fetch("/store/delivery-area", {
    method: "POST",
    body: JSON.stringify(layer.toGeoJSON()    )
});
```

```
fetch("/api/stores/100/delivery-area")
    .then(response => response.json())
    .then(data => {
        L.geoJSON(data)
            .addTo(map);
    });
```

3. Interactive Maps with React

```
import { MapContainer, TileLayer } from "react-leaflet";
import "leaflet/dist/leaflet.css";

function App() {
   const center = [17.3850, 78.4867];

   return (
      <MapContainer
         center={center}
         zoom={13}
         style={{ height: "100vh", width: "100%" }}
      >

         <TileLayer
            url="https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png"
            attribution="&copy; OpenStreetMap contributors"
         />

      </MapContainer>
   );
}
export default App;
```

Understanding the Tile URL

This line is important:

```
<TileLayer url="https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png" />
```

The map is divided into tiles.

tile	tile	tile
tile	tile	tile
tile	tile	tile

The browser downloads only the tiles required for the current viewport.

When the user moves the map:

User drags map
↓
Viewport changes
↓
New tiles required
↓
Browser downloads tiles
↓
Map updates

Zoom Level:

Zoom 1 → World | Zoom 5 → Country | Zoom 10 → City | Zoom 15 → Neighborhood
Zoom 18+ → Street / Building

Adding a Marker

```
import {MapContainer, TileLayer, Marker, Popup} from "react-leaflet";

<Marker position={[17.3850, 78.4867]}>
   <Popup>Kolkata</Popup>
</Marker>
```

Custom Marker Icon

```
const storeIcon = new L.Icon({
   iconUrl: "/store.png",
   iconSize: [40, 40],
   iconAnchor: [20, 40], // margin (left, top)
   popupAnchor: [0, -40]
});
```

Clicking the Map

```
map.on("click", event => {
   console.log(event.latlng.lat);
   console.log(event.latlng.lng);
});
```

Render it:

```
<Marker
   position={[17.3850, 78.4867]}
   icon={storeIcon}
   eventHandlers={{
      click: () => {
         console.log("Store clicked");
      }
   }}
>
   <Popup> My Store  </Popup>
</Marker>
```

Current User Location:

```
navigator.geolocation.getCurrentPosition(
   position => {
      const lat = position.coords.latitude;
      const lng = position.coords.longitude;
      console.log(lat, lng);
   },
   error => { console.error(error);}
);
```

Drawing a Polyline(Line / Road) & Polygon

A polygon is a closed area.

A road can be represented as a sequence of coordinates.

```
const road = [
   [17.3850, 78.4867],
   [17.3860, 78.4880],
   [17.3875, 78.4900],
   [17.3890, 78.4920]
];
```

Render it:

```
import { Polyline, Polygon } from "react-leaflet";

<Polyline positions={road} />

<Polygon positions={deliveryArea} />
```

Drawing Tools

Leaflet itself provides the map rendering. For interactive drawing, Leaflet Draw can be used.

```
Map
 |
 ├── Marker tool
 ├── Polyline tool
 ├── Polygon tool
 ├── Rectangle tool
 ├── Circle tool
 └── Edit/Delete tools
```

Install: `npm install leaflet-draw`

What Happens When the User Draws a Polygon?

Suppose the user clicks:P1 → P2 → P3 → P4 → Finish

Leaflet generates a geometry.

Conceptually:
User interaction
↓
Mouse clicks
↓
Leaflet captures coordinates
↓
Leaflet creates polygon
↓
Polygon layer added to map

You can then extract the coordinates.

```
import "leaflet-draw/dist/leaflet.draw.css";

import { useEffect } from "react";
import L from "leaflet";
import "leaflet-draw";

function DrawingTools({ map, onCreated }) {
   useEffect(() => {
      if (!map) return;
      const drawnItems = new L.FeatureGroup();
      map.addLayer(drawnItems);

      const drawControl = new L.Control.Draw({
         edit: {
            featureGroup: drawnItems
         },
         draw: {
            polygon: true,
            polyline: true,
            rectangle: true,
            circle: true,
            marker: true
         }
      });

      map.addControl(drawControl);

      map.on(L.Draw.Event.CREATED, function (event) {
         const layer = event.layer;
         drawnItems.addLayer(layer);
         onCreated(layer);
      });

      return () => {
         map.removeControl(drawControl);
         map.removeLayer(drawnItems);
      };
   }, [map]);
   return null;
}
```

Getting Polygon Coordinates:

```
const latLngs = layer.getLatLngs();
```

4. GeoJSON - Convert Leaflet Coordinates to API Format

```
const geojson = layer.toGeoJSON();
console.log(geojson);

"geometry": {
      "type": "Polygon",
      "coordinates": [
        [
          [78.4800, 17.3850],
          [78.4900, 17.3950],
          [78.5000, 17.3850],
          [78.4800, 17.3850]
        ]
      ]
  }
```

```
fetch("/store/delivery-area", {
    method: "POST",
    headers: {
        "Content-Type": "application/json"
    },
    body: JSON.stringify(geojson)
});
```

Controller

```
POST /api/stores/{storeId}/delivery-area
```

Controller Request DTO:

```
public record DeliveryAreaRequest(
        Long storeId,
        GeometryDto geometry
) {}
```

```
public record GeometryDto(
        String type,
        List<List<List<Double>>> coordinates
) {}
```

Create PostgreSQL Database

```
CREATE EXTENSION IF NOT EXISTS postgis;
Verify: SELECT PostGIS_Version();

CREATE TABLE delivery_area (
    id BIGSERIAL PRIMARY KEY,
    store_id BIGINT NOT NULL,
    polygon GEOMETRY(POLYGON, 4326) NOT NULL
);
```

Geometry type = Polygon
SRID = 4326

GEOMETRY
GEOGRAPHY
POINT
LINESTRING
POLYGON
MULTIPOLYGON

4326 is the SRID associated with WGS 84 latitude/longitude coordinates.

```
INSERT INTO delivery_area (store_id, geom)
VALUES (101,
ST_GeomFromText(
        'POLYGON((78.4800 17.3850,  78.4900 17.3950, 78.5000 17.3850, 78.4800 17.3850))',
        4326
    )
);
```

Geometry vs Geography

geometry:
Works with planar coordinate calculations.

Good for:
- projected coordinate systems
- many spatial operations
- high-performance geometry calculations

geography:
Designed for geographic coordinates on the Earth.

Useful for:
- latitude/longitude
- distances on Earth's surface
- real-world geographic calculations

Spatial Index

A normal B-tree index is not enough for many spatial queries.

Create a GiST spatial index:

```
CREATE INDEX idx_delivery_area_geom
ON delivery_area
USING GIST (geom);
```

Validate Polygon:

- Polygon Must Be Closed (A → B → C → D → A)
- Minimum Number of Points (at least three distinct points)
- Polygon Self-Intersection (A self-intersecting polygon should be rejected.)

5. Store Polygon Using JTS

JTS can build the polygon:

```
GeometryFactory factory = new GeometryFactory();

Coordinate coordinate = new Coordinate(longitude, latitude);
```

Polygon Builder:

```
public Polygon buildPolygon(List<PointDTO> points) {
    GeometryFactory factory = new GeometryFactory();
    Coordinate[] coordinates = new Coordinate[points.size() + 1];

    for (int i = 0; i < points.size(); i++) {
        PointDTO point = points.get(i);
        coordinates[i] = new Coordinate(point.longitude(), point.latitude());
    }

    coordinates[points.size()] = coordinates[0];
    LinearRing ring = factory.createLinearRing(coordinates);
    return factory.createPolygon(ring);
}
```

Store Polygon Using WKT

```
private String buildWKT(List<List<Double>> coordinates) {
    StringBuilder wkt = new StringBuilder("POLYGON((");

    for (List<Double> coordinate : coordinates) {
        wkt.append(coordinate.get(0))
           .append(" ")
           .append(coordinate.get(1))
           .append(",");
    }

    wkt.setLength(wkt.length() - 1);
    wkt.append("))");
    return wkt.toString();
}
```

PostGIS → GeoJSON

```
SELECT id, store_id, ST_AsGeoJSON(polygon)
FROM delivery_zone
WHERE store_id = ?;
```

PostGIS Queries

Point-in-Polygon Query (ST_Contains)

```
SELECT store_id
FROM delivery_area
WHERE ST_Contains(
   geom,
   ST_SetSRID(
      ST_Point(78.4867, 17.3850),
      4326
   )
);
```

IMPORTANT: ST_Point(longitude, latitude)
NOT ST_Point(latitude, longitude)

```
SELECT store_id FROM delivery_area
WHERE geom && ST_SetSRID(
   ST_Point(78.4867, 17.3850),
   4326
)
AND ST_Contains(
   geom,
   ST_SetSRID(
      ST_Point(78.4867, 17.3850),
      4326
   )
);
```

ST_Within

The reverse relationship can be expressed using **ST_Within(point, polygon)**:

```
SELECT *
FROM delivery_zone
WHERE ST_Within(
    ST_SetSRID(
        ST_Point(78.489, 17.387),
        4326
    ),
    polygon
);
```

ST_Covers

ST_Contains can exclude points lying exactly on the polygon boundary

For delivery eligibility, you may prefer:
ST_Covers(polygon, point)

Calculating Polygon Area

```
SELECT ST_Area( geom::geography
)
FROM delivery_area;
```

The result is in square meters.
Convert to square kilometers:

```
SELECT ST_Area( geom::geography) / 1000000.0
FROM delivery_area;
```

Calculating Distance

```
SELECT ST_Distance(
   point_a::geography,
   point_b::geography
);
```

The result is in meters.

Nearby Store Search

```
SELECT
    store_id,
    ST_Distance(
        location::geography,
        ST_SetSRID(
            ST_Point(78.4867, 17.3850),
            4326
        )::geography
    ) AS distance_meters
FROM store
WHERE ST_DWithin(
    location::geography,
    ST_SetSRID(
        ST_Point(78.4867, 17.3850),
        4326
    )::geography,
    5000
)
ORDER BY distance_meters;
```

Google Maps Equivalent

The concepts remain almost identical.Google Maps provides classes such as:

- google.maps.Map
- google.maps.Marker
- google.maps.Polyline
- google.maps.Polygon
- google.maps.Circle

Leaflet	Google Maps
Marker	Marker APIs
Polyline	Polyline
Polygon	Polygon
Circle	Circle
Map	Map

Google Maps Basic Concept:

```
const map = new google.maps.Map(
   document.getElementById("map"),
   {
      center: {
         lat: 17.3850,
         lng: 78.4867
      },
      zoom: 13
   }
);
```

Google Maps Polyline:

```
const path = [
   { lat: 17.3850, lng: 78.4867 },
   { lat: 17.3860, lng: 78.4880 },
   { lat: 17.3875, lng: 78.4900 }
];

const line = new
google.maps.Polyline({
   path,
   map
});
```

Google Maps Marker Concept:

```
const marker = new
google.maps.marker.AdvancedMarkerElement({
      map,
      position: {
         lat: 17.3850,
         lng: 78.4867
      }
 });
```

Geocoding Vs Reverse Geocoding

Geocoding	Reverse Geocoding
Address ↓ Latitude + Longitude	Latitude + Longitude ↓ Address

Create Spatial Index

> Suppose we have: 10 stores → Checking every polygon is fine.
> But imagine: 1,000,000 stores → Running point-in-polygon against every store is expensive.

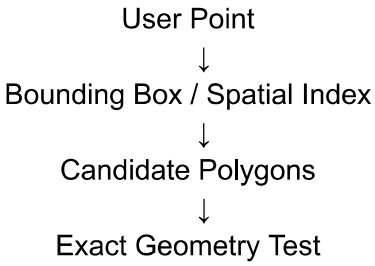
We need an index.

GiST Index:

```
CREATE INDEX idx_delivery_zone_polygon
ON delivery_zone
USING GIST (polygon);
```

Now PostgreSQL can efficiently narrow spatial candidates.

The database can use the spatial index before running the exact geometry test.
Conceptually:



Why Point-In-Polygon Alone Is Not Enough?

Even with PostGIS, the application may need additional high-level filtering depending on the search architecture.

User Location → **GeoHash (17.385, 78.486 → tepg1...)**
→ Nearby Area (**Bounding Box**)
→ KD-Tree
→ Nearest Stores → PostGIS Point-In-Polygon → Deliverable Stores

Store GeoHash References

Possible table:

```
CREATE TABLE delivery_zone_geohash (
    delivery_zone_id BIGINT NOT NULL,
    geohash VARCHAR(20) NOT NULL
);
```

Example:
zone 101 → tepg1
zone 101 → tepg2
zone 101 → tepg3

GeoHash Prefilter

User searches: (17.387, 78.489) **Generate: User GeoHash = tepg1** Search nearby cells: tepg1 tepg2 tepf9 tepg0 ...	Only stores associated with those cells

KD-Tree

GeoHash is useful for grid-based filtering.
KD-Tree is useful for: → Nearest-neighbor search.

Complete Spatial Search

User Location ↓ GeoHash ↓ GeoHash Candidate Stores ↓ KD-Tree ↓ Nearest Stores ↓ PostGIS ↓ Point-In-Polygon ↓ Supported Stores	STEP 1 Generate GeoHash ↓ tepg1 STEP 2 Find candidate stores ↓ 50 stores STEP 3 KD-Tree ↓ 20 nearest STEP 4 Point-In-Polygon ↓ 8 deliverable STEP 5 Return stores

Polygon Simplification

Large polygons may contain thousands of points.

Example:

> **10,000 vertices**

This creates:.

- Large payloads
- More storage
- More processing
- Slower rendering

We can simplify the geometry.

Solution: Douglas-Peucker Algorithm

Dynamic Delivery Zones

Instead of permanently storing: **Polygon A**
we can calculate: Effective Delivery Area
Using:

Base Polygon + Traffic + Driver Availability + Weather + Demand	**Traffic-Aware Zones** **Example:** Traffic = Low ⇒ Delivery Radius: 10 km Traffic = Severe ⇒ Effective Delivery Radius: 5 km **For polygon systems:** Base Polygon ↓ Traffic-aware adjustment ↓ Effective Polygon

Driver Availability

Suppose: **Drivers available = 20**
Delivery zone: **Large**
But: **Drivers available = 3**

The system may reduce the active delivery region.

This should generally be implemented as a business rule or optimization layer around the base polygon rather than modifying the owner's original polygon.

Weather-Aware Zones

Heavy rain may increase:

- Travel time
- Driver risk
- ETA

The system can temporarily reduce the effective zone.

Base Polygon
+
Weather
+
Traffic
↓
Effective Delivery Area

Demand-Aware Zones

During high demand:
Orders ↑
Drivers ↓
the system can reduce the active service area.

This connects the delivery-zone system with:

- Demand Prediction
- Driver Supply
- Surge Pricing
- ETA

Delivery Fee by Polygon

Different polygons can have different delivery fees.

> Polygon A: 0–3 km ⇒ ₹20
> Polygon B: 3–6 km ⇒ ₹40
> Polygon C: 6–10 km ⇒ ₹60

Therefore polygon data can also drive pricing.

Minimum Order by Polygon

Business rules can vary geographically.

> Zone A: Minimum order = ₹199
> Zone B: Minimum order = ₹299

Polygon-Based Surge Pricing

The delivery zone can also integrate with surge pricing.

> **Polygon A** : Demand = High | Driver Supply = Low
>
> **Pricing engine**: Polygon + Demand + Driver Supply + Traffic ⇒ Surge Multiplier

Polygon versioning

Do not always overwrite the old polygon without tracking history.

> Version 1: 10 km zone
> Version 2: 8 km zone
> Version 3: 6 km zone

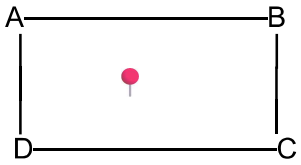
> **Database;**
> delivery_zone
> delivery_zone_version

How a system like JTS/PostGIS actually works internally

```
class Point {
    double x;
    double y;
}
```

```
class Polygon {
    List<Point> points;
}
```

1. Point in a Polygon



boolean **contains**(Polygon polygon, Point point)

⇒ The classic algorithm is the **Ray Casting algorithm**

2. Orientation / Cross Product

This is one of the most important primitives in computational geometry.

Given:

```
A
 \
  \
   B
    \
     C
```

We want to know whether: A → B → C

turns:
left
right
or is collinear

Use the cross product:

```
cross =
(B.x - A.x) * (C.y - A.y)
-
(B.y - A.y) * (C.x - A.x)
```

Result:
> 0 → counter-clockwise
< 0 → clockwise
= 0 → collinear

3. Line Segment Intersection

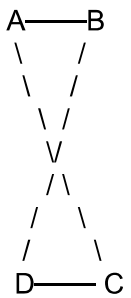
boolean intersects(Point a, Point b, Point c, Point d)

This becomes the foundation for:

Polygon intersectionRoad intersection
Self-intersection detection

4. Polygon Self-Intersection

Every geometry can have a bounding box:



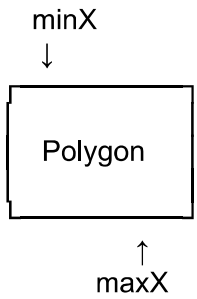
A polygon can accidentally cross itself.

You can detect this by comparing polygon edges:
edge 1 vs edge 2
edge 1 vs edge 3
edge 1 vs edge 4
…

Naively: $O(n^2)$
This becomes expensive for large polygons. ⇒ That leads us to spatial indexes.

5. Bounding Box

Every geometry can have a bounding box:



Store:

```
class Envelope {
    double minX;
    double minY;
    double maxX;
    double maxY;
}
```

Then you can quickly ask:
Could these objects possibly intersect?

6. Distance Between Two Points

For Earth distances, you can use: **Haversine**

7. Polygon Area

For simple Cartesian coordinates, you can use the **Shoelace formula.**

8. Polygon Intersection

9. Polygon Union

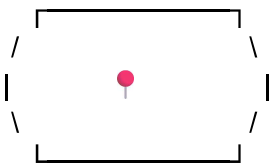
10. Polygon Difference

11. Buffer

Another interesting operation is:

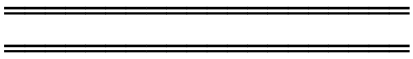
```
buffer(point, 5km)
```

Conceptually:



For a line:

a buffer creates:



This is useful for:

- road corridors
- service radius
- Geofencing

Implementing a robust buffer operation yourself is substantially harder.

11. Topology

This is where a serious geometry engine becomes complicated.

```
Contains
Within
Intersects
Touches
Crosses
Overlaps
Disjoint
Equals
```

For example:

- A contains B
- A intersects B
- A touches B
- A overlaps B

These are called **topological predicates**.

Why JTS Is Much More Than Point-in-Polygon

A simplified homemade geometry library might look like:

```
MyGeometryEngine
├── Point
├── LineString
├── Polygon
├── Envelope
├── Distance
├── Orientation
├── SegmentIntersection
├── PointInPolygon
├── PolygonValidation
└── SpatialIndex
```

A serious geometry engine eventually needs:

```
MyGeometryEngine
├── Geometry Model
├── Precision Model
├── Coordinate Systems
├── Robust Predicates
├── Segment Intersection
├── Polygon Overlay
├── Union
├── Difference
├── Intersection
├── Buffer
├── Simplification
├── Validation
├── Spatial Indexes
├── Prepared Geometry
└── Topology
```

That's why mature geometry libraries are large projects.

Precision Is a Huge Problem:

Suppose mathematically:

```
A
 \
  \
   B
```

Two lines should intersect exactly at:
x = 10.000000000

But floating-point calculations might produce:
9.999999999999998

This can cause topology bugs. Therefore geometry libraries need robust handling of:

- floating-point precision
- rounding
- numerical robustness
- near-collinear points
- near-touching polygons

This is one of the reasons a simple implementation can work for demonstrations but fail on real-world geographic data.